МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет прикладной математики и информатики
Кафедра многопроцессорных систем и сетей

Сергиенко Лев Эдуардович

Отчет по “Лабораторная работа 5”

Преподаватели
Волчецкая П.С.
Дрепакова А.В.

Минск, 2025

Формулировка проблемы

ВАРИАНТ 20

1. Построить сетевой график для максимальной ($t_{пес}$) продолжительности всех его работ, рассчитать наиболее ранние и наиболее поздние сроки наступления событий, найти критический путь, определить полные и независимые резервы времени всех работ и коэффициенты напряженности некритических дуг.

2. Для трехпараметрической модели найти ожидаемое время выполнения проекта, определить вероятность выполнения проекта не позднее заданного срока, найти интервал гарантированного (с вероятностью $P = 0{,}9973$) времени выполнения проекта, оценить максимально возможный срок выполнения проекта с заданной надежностью.

Выполнить те же расчеты для двухпараметрической модели. Сравнить результаты.

3. Считая $t_{пес}$ продолжительностью работы с минимальной допустимой интенсивностью ($t_{пес} = t_{max}$), а $t_{опт}$ – продолжительностью работы с максимальной возможной интенсивностью ($t_{опт} = t_{min}$), найти оптимальный по стоимости вариант выполнения проекта.

Минимизировать стоимость проекта при минимально возможном сроке его исполнения.

Работа	Опирается на работы	$t_{пес}$	$t_{вер}$	$t_{опт}$	Стоимость сокращения работы на один день, s_k
b_1	–	8	6	3	4
b_2	–	6	4	2	8
b_3	b_1	13	10	6	5
b_4	b_2	4	2	1	6
b_5	b_2	5	4	2	7
b_6	b_3, b_4	10	9	4	4
b_7	b_5	6	3	1	10
b_8	b_5, b_6	9	4	2	9
b_9	b_5, b_6	10	6	3	5
b_{10}	b_7, b_8	7	5	3	8
b_{11}	b_9, b_{10}	11	9	5	7

Директивный (заданный) срок выполнения проекта $T_{дир} = 45$ дней.
Заданная надежность $\gamma = 0{,}95$.
Стоимость одного дня проекта равна 10 денежным единицам: $S = 10$.

Математическая модель

Задача 1. Метод критического пути

Работа – процесс, требующий затрат времени и ресурсов. Обозначается стрелкой (дугой графа).

Событие – момент завершения работы или группы работ. Обозначается кружком (вершиной графа).

Фиктивная работа – логическая связь между работами, не требующая затрат времени и ресурсов (продолжительность = 0). Изображается пунктирной стрелкой.

Критический путь – путь максимальной длины от начального до конечного события.

Правила построения сетевого графика

1. Одно исходное и одно завершающее событие
2. Любые два события соединены не более чем одной работой
3. Отсутствие циклов и петель
4. Нет "хвостов" и "тупиков"

Формулы расчета временных параметров

Ранние сроки наступления событий

$$T_p(i) = \max_{j \in P(i)} T_p(j) + t_{ji}$$

где $P(i)$ – множество предшественников события i

Начальное условие: $T_p(0) = 0$

Поздние сроки наступления событий

$$T_n(i) = \min_{j \in S(i)} T_n(j) - t_{ij}$$

где (i) – множество последователей события i

Граничное условие: $T_n(n) = T_{kp}$ (для конечного события)

Критическое время проекта

$T_{kp} = T_p(n)$

где n – конечное событие

Резервы времени

Резерв времени события

$R(i) = T_n(i) - T_p(i)$

Критические события: $R(i) = 0$

Полный резерв времени работы

$r_n(b_k) = r_n(i,j) = T_n(j) - T_p(i) - t_{ij}$

или

$r_n(b_k) = E_n(b_k) - S_p(b_k) - t_{ij}$

где:

- $S_p(b_k) = T_p(i)$ – наиболее ранний срок начала работы
- $E_n(b_k) = T_n(j)$ – наиболее поздний срок окончания работы

Независимый (свободный) резерв времени работы

$r_H(b_k) = r_H(i,j) = T_p(j) - T_n(i) - t_{ij}$

Критические работы: $r_n(b_k) = 0$ и $r_H(b_k) = 0$

Условие критического пути

Работа (i,j) лежит на критическом пути, если:

$T_p(i) = T_n(i) \ \# \ T_p(j) = T_n(j) \ \# \ T_n(j) - T_p(i) - t_{ij} = 0$

Анализ напряженности некритических дуг

Резерв времени некритической дуги

$$R(b) = a - b$$

где:

- a – длина замыкающего критического участка
- b – длина некритической дуги

Коэффициент напряженности

$$N(b) = \frac{b}{a} = 1 - \frac{R(b)}{a}$$

Классификация по напряженности

- **Критическая зона:** $N(b) > 0.8$
- **Подкритическая зона:** $0.6 \leq N(b) \leq 0.8$
- **Резервная зона:** $N(b) < 0.6$

Задача 2. Вероятностные характеристики сетевых планов

Трехпараметрическая модель

Ожидаемая продолжительность работы

$$t_{\text{oj}} = \frac{t_{\text{opt}} + 4t_{\text{ver}} + t_{\text{pes}}}{6}$$

где:

- t_{opt} – оптимистическое время
- t_{ver} – наиболее вероятное время
- t_{pes} – пессимистическое время

Дисперсия продолжительности работы

$$\sigma^2(t_{\text{oj}}) = \left(\frac{t_{\text{pes}} - t_{\text{opt}}}{6}\right)^2$$

Двухпараметрическая модель

Ожидаемая продолжительность работы

$$t^*\text{oj} = \frac{2t\text{opt} + 3t_{\text{pes}}}{5}$$

Дисперсия

$$\sigma^2(t^*\text{oj}) = \left(\frac{t\text{pes} - t_{\text{opt}}}{6}\right)^2$$

(та же формула, что в трехпараметрической модели)

Характеристики критического пути

Дисперсия критического пути

$$\sigma^2_{\text{kp}} = \sum_{i \in KP} \sigma_i^2$$

где суммирование по всем работам критического пути

Среднеквадратическое отклонение

$$\sigma_{\text{kp}} = \sqrt{\sigma^2_{\text{kp}}}$$

Вероятностный анализ

Вероятность выполнения к директивному сроку

$$P(t_{\text{kp}} \le T_{\text{dir}}) = 0.5 + \Phi\left(\frac{T_{\text{dir}} - T_{\text{kp}}}{\sigma_{\text{kp}}}\right)$$

где $\Phi(x)$ – функция Лапласа:

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_0^x e^{-\frac{t^2}{2}} dt$$

Правило трех сигм (интервал гарантированного выполнения)

$$P(T_{\text{кр}} - 3\sigma_{\text{кр}} \le t \le T_{\text{кр}} + 3\sigma_{\text{кр}}) \approx 0.9973$$

Максимальный срок с заданной надежностью γ

Метод 1 (симметричный интервал):

$$P(|t_{\text{кр}} - T_{\text{кр}}| \le z_\gamma \sigma_{\text{кр}}) = 2\Phi(z_\gamma) = \gamma$$

откуда $\Phi(z_\gamma) = \gamma/2$

Метод 2 (односторонняя оценка):

$$P(t_{\text{кр}} \le T) = 0.5 + \Phi(z_\gamma) = \gamma$$

откуда $\Phi(z_\gamma) = \gamma - 0.5$

$$T = T_{\text{кр}} + z_\gamma \sigma_{\text{кр}}$$

Свойства функции Лапласа

- $\Phi(-x) = -\Phi(x)$ (нечетная функция)
- $\Phi(x) \approx 0.5$ при $x \ge 5$
- $\Phi(1.65) = 0.45 \rightarrow \gamma = 0.95$
- $\Phi(1.96) = 0.475 \rightarrow \gamma = 0.95$ (двусторонний)

Задача 3. Методы оптимизации стоимости сетевых проектов

- $t_{\max} = t_{pes}$ – максимальная (нормальная) продолжительность работы с минимальной интенсивностью
- $t_{\min} = t_{opt}$ – минимальная продолжительность работы с максимальной интенсивностью
- s_k – стоимость сокращения работы k на один день
- S – стоимость одного дня проекта

Параметры сокращения работ

Резерв сокращения работы

$$t_k^c = t_{\max} - t_{\min}$$

количество дней, на которое можно сократить работу k

Экономия от сокращения работы

$$\Delta_k = S - s_k$$

где:

- если $\Delta_k > 0$ – стоимость проекта уменьшается
- если $\Delta_k < 0$ – стоимость проекта увеличивается

Суммарная экономия

$$\Sigma\Delta_k = \Delta_k \cdot t_k^c$$

Общая стоимость проекта

При максимальных продолжительностях

$$S(t_{\max}) = T_{cr} \cdot S$$

При минимальных продолжительностях

$$S(t_{\min}) = T_{\min} \cdot S + S_{add}$$

$$S_{add} = \sum_k s_k \cdot t_k^c$$

где – дополнительные затраты на сокращение всех работ

Алгоритм оптимизации (прямой ход)

Выбор времени сокращения на шаге i

$$t^c = \min R_k^i, R(b)$$

где:

- R_k^i – резерв сокращения работы на шаге i
- $R(b)$ – резерв некритической дуги

Изменение стоимости на шаге

$$S^i = S^{i-1} - t^c \cdot \Delta^i$$

где $\Delta^i = S - \Sigma s_k$ для выбранных работ

Алгоритм оптимизации (обратный ход)

Начинаем с минимальных продолжительностей всех работ, затем увеличиваем продолжительности некритических работ.

Приоритет увеличения

Увеличивать в первую очередь работы с **максимальной** стоимостью сокращения s_k

Условие остановки

Процесс завершается, когда хотя бы один критический путь состоит только из работ с минимальной продолжительностью

Резервы некритических дуг

Для дуги, проходящей через работу

$$R(i,j) = r_p(i,j) = T_p(j) - T_r(i) - t_{ij}$$

Для дуги между критическими событиями

$$R(a,b) = T_p(b) - T_r(a) - L(a,b)$$

где $L(a,b)$ – длина дуги

Критерии выбора работ для сокращения

Для параллельных критических путей

На каждом пути выбирается работа с **минимальной** стоимостью сокращения s_k

Оптимальная комбинация

Выбирается комбинация работ с минимальной суммарной стоимостью:

$$\min \sum s_k$$

Дополнительные затраты на шаге

$$S_{add}^{i} = t^{c} \cdot \sum s_k$$

где t^{c} – время сокращения на данном шаге

Решение

```
import networkx as nx
import numpy as np
from scipy.stats import norm

# Определяем связи между точками
edges_points = {
    'b1(8)': (1, 2),
    'b2(6)': (1, 3),
    'b3(13)': (2, 4),
    'b4(4)': (3, 4),
    'b5(5)': (3, 5),
    'b6(10)': (4, 6),
    'b7(6)': (5, 7),
    'b8(9)': (6, 8),
    'b9(10)': (6, 9),
    'b10(7)': (7, 10),
    'b11(11)': (9, 11)
}

# Веса работ
weights = {
    'b1(8)': 8,
    'b2(6)': 6,
    'b3(13)': 13,
    'b4(4)': 4,
    'b5(5)': 5,
    'b6(10)': 10,
    'b7(6)': 6,
    'b8(9)': 9,
    'b9(10)': 10,
    'b10(7)': 7,
    'b11(11)': 11
}
```

```
# Фиктивные работы для правильных зависимостей
fictitious_edges = [
    (5, 8, 'f1', 0),
    (5, 9, 'f2', 0),
    (8, 10, 'f3', 0),
    (10, 11, 'f4', 0)
]

# Директивный срок
T_dir = 45

gamma = 0.95
S = 10  # Стоимость одного дня проекта

pert_data = {
    'b1': {'t_pes': 8, 't_ver': 6, 't_opt': 3},
    'b2': {'t_pes': 6, 't_ver': 4, 't_opt': 2},
    'b3': {'t_pes': 13, 't_ver': 10, 't_opt': 6},
    'b4': {'t_pes': 4, 't_ver': 2, 't_opt': 1},
    'b5': {'t_pes': 5, 't_ver': 4, 't_opt': 2},
    'b6': {'t_pes': 10, 't_ver': 9, 't_opt': 4},
    'b7': {'t_pes': 6, 't_ver': 3, 't_opt': 1},
    'b8': {'t_pes': 9, 't_ver': 4, 't_opt': 2},
    'b9': {'t_pes': 10, 't_ver': 6, 't_opt': 3},
    'b10': {'t_pes': 7, 't_ver': 5, 't_opt': 3},
    'b11': {'t_pes': 11, 't_ver': 9, 't_opt': 5}
}


# Данные для оптимизации стоимости
t_pes_opt = {
    (1, 2): 8, (1, 3): 6, (2, 4): 13, (3, 4): 4,
    (3, 5): 5, (4, 6): 10, (5, 7): 6, (6, 8): 9,
    (6, 9): 10, (7, 10): 7, (9, 11): 11
}

t_opt_opt = {
    (1, 2): 3, (1, 3): 2, (2, 4): 6, (3, 4): 1,
    (3, 5): 2, (4, 6): 4, (5, 7): 1, (6, 8): 2,
    (6, 9): 3, (7, 10): 3, (9, 11): 5
}

# Стоимость сокращения на 1 день
costs = {
    (1, 2): 4, (1, 3): 8, (2, 4): 5, (3, 4): 6,
    (3, 5): 7, (4, 6): 4, (5, 7): 10, (6, 8): 9,
    (6, 9): 5, (7, 10): 8, (9, 11): 7
}

def build_network_graph(edges_points, weights, fictitious_edges):
```

```
    """
    Построение сетевого графа проекта

    Параметры:
    - edges_points: словарь связей между узлами
    - weights: словарь весов работ
    - fictitious_edges: список фиктивных работ

    Возвращает:
    - G: направленный граф NetworkX
    """

    G = nx.DiGraph()

    # Добавляем основные работы
    for edge, points in edges_points.items():
        G.add_edge(points[0], points[1], label=edge, weight=weights[edge])

    # Добавляем фиктивные работы
    for start, end, label, weight in fictitious_edges:
        G.add_edge(start, end, label=label, weight=weight)

    return G


# ============================================================================
# АНАЛИЗ КРИТИЧЕСКОГО ПУТИ
# ============================================================================

def calculate_earliest_times(G):
    earliest_start = {node: 0 for node in G.nodes()}

    for node in nx.topological_sort(G):
        for pred in G.predecessors(node):
            earliest_start[node] = max(
                earliest_start[node],
                earliest_start[pred] + G.edges[pred, node]['weight']
            )

    return earliest_start


def calculate_latest_times(G, earliest_start):
    final_node = max(G.nodes())
    latest_start = {node: float('inf') for node in G.nodes()}
    latest_start[final_node] = earliest_start[final_node]

    for node in reversed(list(nx.topological_sort(G))):
        for succ in G.successors(node):
            if latest_start[succ] != float('inf'):
                latest_start[node] = min(
```

```
                    latest_start[node],
                    latest_start[succ] - G.edges[node, succ]['weight']
                )

    return latest_start


def find_critical_path(G, earliest_start, latest_start):
    critical_path_edges = []

    for u, v in G.edges():
        if G[u][v]['weight'] > 0:  # Только основные работы
            if (earliest_start[u] == latest_start[u] and
                earliest_start[v] == latest_start[v]):
                if latest_start[v] - earliest_start[u] - G[u][v]['weight'] == 0:
                    critical_path_edges.append((u, v))

    critical_path_nodes = [
        node for node in G.nodes()
        if earliest_start[node] == latest_start[node]
    ]

    return critical_path_edges, critical_path_nodes


def calculate_floats(G, earliest_start, latest_start):
    total_float = {}
    free_float = {}

    for u, v in G.edges():
        if G[u][v]['weight'] > 0:  # Только основные работы
            total_float[(u, v)] = (latest_start[v] - earliest_start[u] -
                                   G[u][v]['weight'])
            free_float[(u, v)] = (earliest_start[v] - earliest_start[u] -
                                  G[u][v]['weight'])

    return total_float, free_float


def calculate_tension_coefficients(total_float, earliest_start, latest_start):
    tension_coeff = {}

    for edge in total_float:
        if total_float[edge] > 0:  # Только некритические
            tension_coeff[edge] = 1 - (
                total_float[edge] /
                (latest_start[edge[1]] - earliest_start[edge[0]])
            )

    return tension_coeff
```

```
# ============================================================================
# ТРЕХПАРАМЕТРИЧЕСКАЯ МОДЕЛЬ
# ============================================================================

def calculate_pert_3param(pert_data):
    t_expected = {}
    variance = {}

    for activity, data in pert_data.items():
        # t_ож = (t_опт + 4*t_вер + t_пес)/6
        t_exp = (data['t_opt'] + 4 * data['t_ver'] + data['t_pes']) / 6
        t_expected[activity] = round(t_exp, 2)

        # σ² = ((t_пес - t_опт)/6)^2
        var = ((data['t_pes'] - data['t_opt']) / 6) ** 2
        variance[activity] = round(var, 2)

    return t_expected, variance


def calculate_critical_path_stats_3param(critical_works, t_expected, variance):
    """
    Расчет характеристик критического пути для 3-параметрической модели

    Параметры:
    - critical_works: список критических работ
    - t_expected: словарь ожидаемых времен
    - variance: словарь дисперсий

    Возвращает:
    - T_expected: ожидаемое время выполнения проекта
    - total_variance: дисперсия критического пути
    - std_dev: среднеквадратическое отклонение
    """

    total_variance = 0
    T_expected = 0

    for activity in critical_works:
        act_name = activity.split('(')[0]
        if act_name in variance:
            total_variance += variance[act_name]
            T_expected += t_expected[act_name]

    std_dev = np.sqrt(total_variance)

    return T_expected, total_variance, std_dev


def calculate_probabilities_3param(T_expected, std_dev, gamma=0.95):
```

```
    """
    Вероятностный анализ для трехпараметрической модели

    Параметры:
    - T_expected: ожидаемое время выполнения
    - std_dev: среднеквадратическое отклонение
    - gamma: требуемая надежность

    Возвращает:
    - probability: вероятность выполнения к ожидаемому сроку
    - lower_bound: нижняя граница интервала (3σ)
    - upper_bound: верхняя граница интервала (3σ)
    - T_max: максимальный срок с заданной надежностью
    """

    if std_dev > 0:
        z_score = (T_expected - T_expected) / std_dev
        probability = norm.cdf(z_score)
    else:
        probability = 1.0

    # Интервал 3σ
    interval_3sigma = 3 * std_dev
    lower_bound = max(0, T_expected - interval_3sigma)
    upper_bound = T_expected + interval_3sigma

    # Максимальный срок с заданной надежностью
    z_gamma = norm.ppf(gamma)
    T_max = T_expected + z_gamma * std_dev

    return probability, lower_bound, upper_bound, T_max


# ============================================================================
# ДВУХПАРАМЕТРИЧЕСКАЯ МОДЕЛЬ
# ============================================================================

def calculate_pert_2param(pert_data):
    t_expected = {}
    variance = {}

    for activity, data in pert_data.items():
        # t*_ож = (2*t_опт + 3*t_пес)/5
        t_exp = (2 * data['t_opt'] + 3 * data['t_pes']) / 5
        t_expected[activity] = round(t_exp, 2)

        # Дисперсия такая же, как в трехпараметрической
        var = ((data['t_pes'] - data['t_opt']) / 6) ** 2
        variance[activity] = round(var, 2)

    return t_expected, variance
```

```python
def calculate_critical_path_stats_2param(critical_works, t_expected, variance):
    """
    Расчет характеристик критического пути для 2-параметрической модели

    Параметры:
    - critical_works: список критических работ
    - t_expected: словарь ожидаемых времен
    - variance: словарь дисперсий

    Возвращает:
    - T_expected: ожидаемое время выполнения проекта
    - total_variance: дисперсия критического пути
    - std_dev: среднеквадратическое отклонение
    """

    total_variance = 0
    T_expected = 0

    for activity in critical_works:
        act_name = activity.split('(')[0]
        if act_name in variance:
            total_variance += variance[act_name]
            T_expected += t_expected[act_name]

    std_dev = np.sqrt(total_variance)

    return T_expected, total_variance, std_dev


def calculate_probabilities_2param(T_expected, std_dev, gamma=0.95):
    """
    Вероятностный анализ для двухпараметрической модели

    Параметры:
    - T_expected: ожидаемое время выполнения
    - std_dev: среднеквадратическое отклонение
    - gamma: требуемая надежность

    Возвращает:
    - probability: вероятность выполнения к ожидаемому сроку
    - lower_bound: нижняя граница интервала (3σ)
    - upper_bound: верхняя граница интервала (3σ)
    - T_max: максимальный срок с заданной надежностью
    """

    if std_dev > 0:
        z_score = (T_expected - T_expected) / std_dev
        probability = norm.cdf(z_score)
    else:
```

```
        probability = 1.0

    # Интервал 3σ
    interval_3sigma = 3 * std_dev
    lower_bound = max(0, T_expected - interval_3sigma)
    upper_bound = T_expected + interval_3sigma

    # Максимальный срок с заданной надежностью
    z_gamma = norm.ppf(gamma)
    T_max = T_expected + z_gamma * std_dev

    return probability, lower_bound, upper_bound, T_max


# ============================================================================
# ОПТИМИЗАЦИЯ СРОКОВ И СТОИМОСТИ
# ============================================================================

def build_optimization_graph(t_pes, costs):
    G = nx.DiGraph()

    for edge, duration in t_pes.items():
        G.add_edge(edge[0], edge[1], weight=duration, cost=costs[edge])

    return G


def calculate_critical_path_optimization(graph):
    """
    Расчет критического пути для графа оптимизации

    Параметры:
    - graph: направленный граф

    Возвращает:
    - earliest: словарь ранних сроков
    - latest: словарь поздних сроков
    - critical_edges: список критических дуг
    - critical_nodes: список критических узлов
    """

    # Ранние сроки
    earliest = {node: 0 for node in graph.nodes()}
    for node in nx.topological_sort(graph):
        for pred in graph.predecessors(node):
            earliest[node] = max(
                earliest[node],
                earliest[pred] + graph.edges[pred, node]['weight']
            )

    # Поздние сроки
```

```
    latest = {node: float('inf') for node in graph.nodes()}
    latest[max(graph.nodes())] = earliest[max(graph.nodes())]

    for node in reversed(list(nx.topological_sort(graph))):
        for succ in graph.successors(node):
            latest[node] = min(
                latest[node],
                latest[succ] - graph.edges[node, succ]['weight']
            )

    # Критический путь
    critical_nodes = [
        node for node in graph.nodes()
        if earliest[node] == latest[node]
    ]

    critical_edges = []
    for u, v in graph.edges():
        if (earliest[u] == latest[u] and
            earliest[v] == latest[v] and
            latest[v] - earliest[u] - graph.edges[u, v]['weight'] == 0):
            critical_edges.append((u, v))

    return earliest, latest, critical_edges, critical_nodes


def optimize_project_cost(G_opt, t_pes, t_opt, costs, T_dir, S):
    """
    Оптимизация стоимости проекта путем сокращения работ

    Параметры:
    - G_opt: граф для оптимизации
    - t_pes: пессимистические оценки
    - t_opt: оптимистические оценки
    - costs: стоимости сокращения
    - T_dir: директивный срок
    - S: стоимость одного дня проекта

    Возвращает:
    - total_cost_reduction: общая стоимость сокращения
    - current_duration: итоговая продолжительность
    - reductions_applied: словарь примененных сокращений
    - optimization_steps: список шагов оптимизации
    """

    # Максимальное сокращение
    max_reduction = {edge: t_pes[edge] - t_opt[edge] for edge in t_pes}

    # Исходный критический путь
    earliest_orig, latest_orig, critical_edges_orig, _ = \
        calculate_critical_path_optimization(G_opt)
```

```
T_original = earliest_orig[max(G_opt.nodes())]

# Сортировка работ по стоимости сокращения
sorted_edges_by_cost = sorted(costs.keys(), key=lambda edge: costs[edge])

total_cost_reduction = 0
current_duration = T_original
reductions_applied = {}
optimization_steps = []

# Процесс оптимизации
while current_duration > T_dir:
    reduction_made = False

    for edge in sorted_edges_by_cost:
        if (max_reduction[edge] > 0 and
            edge in critical_edges_orig and
            current_duration > T_dir):

            # Определяем возможное сокращение
            possible_reduction = min(
                max_reduction[edge],
                current_duration - T_dir
            )

            if possible_reduction > 0:
                # Применяем сокращение
                G_opt.edges[edge]['weight'] -= possible_reduction
                max_reduction[edge] -= possible_reduction
                step_cost = costs[edge] * possible_reduction
                total_cost_reduction += step_cost
                current_duration -= possible_reduction

                # Сохраняем информацию
                if edge not in reductions_applied:
                    reductions_applied[edge] = 0
                reductions_applied[edge] += possible_reduction

                optimization_steps.append({
                    'edge': edge,
                    'reduction': possible_reduction,
                    'cost': step_cost,
                    'duration': current_duration
                })

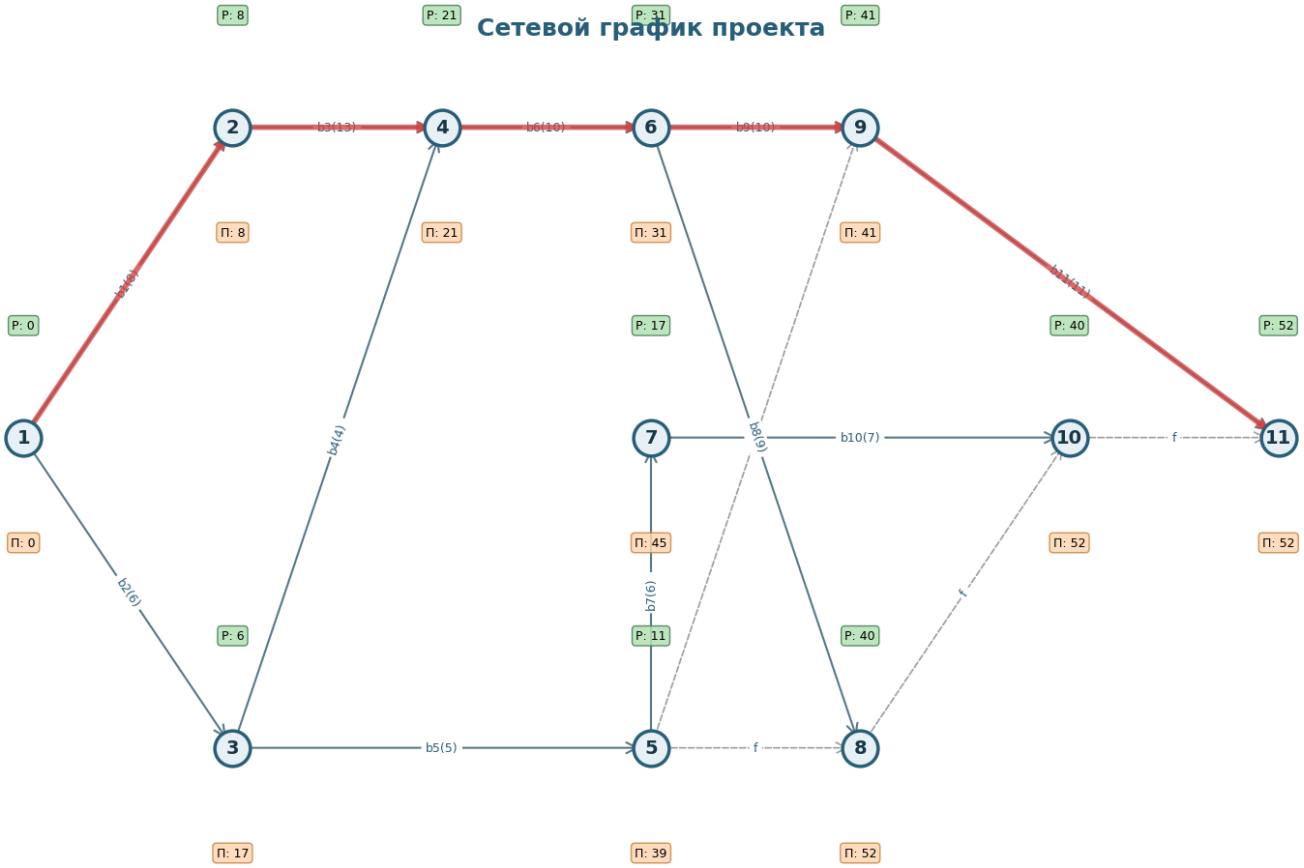
                reduction_made = True

                # Пересчитываем критический путь
                earliest_orig, latest_orig, critical_edges_orig, _ = \
                    calculate_critical_path_optimization(G_opt)
                break
```

```
            if not reduction_made:
                break

    return total_cost_reduction, current_duration, reductions_applied,
optimization_steps
```

Результат



АНАЛИЗ СЕТЕВОГО ГРАФИКА

▸ Критическое время проекта: 52 дней

┌─ Ранние сроки событий
│ Событие 1 → 0 дней
│ Событие 2 → 8 дней
│ Событие 3 → 6 дней
│ Событие 4 → 21 дней
│ Событие 5 → 11 дней
│ Событие 6 → 31 дней

│ Событие 7 → 17 дней
│ Событие 8 → 40 дней
│ Событие 9 → 41 дней
│ Событие 10 → 40 дней
│ Событие 11 → 52 дней

┌─ Поздние сроки событий
│ Событие 1 →   0 дней
│ Событие 2 →   8 дней
│ Событие 3 → 17 дней
│ Событие 4 → 21 дней
│ Событие 5 → 39 дней
│ Событие 6 → 31 дней
│ Событие 7 → 45 дней
│ Событие 8 → 52 дней
│ Событие 9 → 41 дней
│ Событие 10 → 52 дней
│ Событие 11 → 52 дней

▸ Критические события: 1, 2, 4, 6, 9, 11

┌─ Критические работы
│ • b1(8)
│ • b3(13)
│ • b6(10)
│ • b9(10)
│ • b11(11)

┌─ Полные резервы времени
│ (1 → 2) : [КРИТИЧЕСКАЯ]
│ (1 → 3) : резерв 11 дн.
│ (2 → 4) : [КРИТИЧЕСКАЯ]
│ (3 → 4) : резерв 11 дн.
│ (3 → 5) : резерв 28 дн.
│ (4 → 6) : [КРИТИЧЕСКАЯ]
│ (5 → 7) : резерв 28 дн.
│ (6 → 8) : резерв 12 дн.
│ (6 → 9) : [КРИТИЧЕСКАЯ]
│ (7 → 10) : резерв 28 дн.
│ (9 → 11) : [КРИТИЧЕСКАЯ]

┌─ Свободные резервы времени
│ (1 → 2) : 0 дн.
│ (1 → 3) : 0 дн.
│ (2 → 4) : 0 дн.
│ (3 → 4) : 11 дн.
│ (3 → 5) : 0 дн.
│ (4 → 6) : 0 дн.

│ (5 → 7) : 0 дн.
│ (6 → 8) : 0 дн.
│ (6 → 9) : 0 дн.
│ (7 → 10) : 16 дн.
│ (9 → 11) : 0 дн.

┌─ Коэффициенты напряженности
│ (1 → 3) : 0.353 [РЕЗЕРВНАЯ]
│ (3 → 4) : 0.267 [РЕЗЕРВНАЯ]
│ (3 → 5) : 0.152 [РЕЗЕРВНАЯ]
│ (5 → 7) : 0.176 [РЕЗЕРВНАЯ]
│ (6 → 8) : 0.429 [РЕЗЕРВНАЯ]
│ (7 → 10) : 0.200 [РЕЗЕРВНАЯ]

Директивный срок : 45 дней
Расчетный срок : 52 дней
Статус : Превышение срока на 7 дн.

ТРЕХПАРАМЕТРИЧЕСКАЯ МОДЕЛЬ

Работа	t_пес	t_вер	t_опт	t_ож	σ²
b1	8	6	3	5.83	0.69
b2	6	4	2	4.00	0.44
b3	13	10	6	9.83	1.36
b4	4	2	1	2.17	0.25
b5	5	4	2	3.83	0.25
b6	10	9	4	8.33	1.00
b7	6	3	1	3.17	0.69
b8	9	4	2	4.50	1.36
b9	10	6	3	6.17	1.36
b10	7	5	3	5.00	0.44
b11	11	9	5	8.67	1.00

ДВУХПАРАМЕТРИЧЕСКАЯ МОДЕЛЬ PERT

Работа	t_пес	t_опт	t*_ож	σ²
b1	8	3	6.00	0.69
b2	6	2	4.40	0.44

b3	13	6	10.20	1.36
b4	4	1	2.80	0.25
b5	5	2	3.80	0.25
b6	10	4	7.60	1.00
b7	6	1	4.00	0.69
b8	9	2	6.20	1.36
b9	10	3	7.20	1.36
b10	7	3	5.40	0.44
b11	11	5	8.60	1.00

ВЕРОЯТНОСТНЫЙ АНАЛИЗ (3-параметрическая)

- Ожидаемое время T_ож : 38.83 дней
- Дисперсия σ²_кр : 5.41
- СКО σ_кр : 2.33 дней

- Вероятность к сроку 38.83 : 50.00%
- Интервал 3σ (P=0.9973) : [31.9 — 45.8] дней
- Макс. срок (γ=0.95) : 42.7 дней

ВЕРОЯТНОСТНЫЙ АНАЛИЗ (2-параметрическая)

- Ожидаемое время T*_ож : 39.60 дней
- Дисперсия σ²_кр : 5.41
- СКО σ_кр : 2.33 дней

- Вероятность к сроку 39.60 : 50.00%
- Интервал 3σ (P=0.9973) : [32.6 — 45.8] дней
- Макс. срок (γ=0.95) : 43.4 дней

ОПТИМИЗАЦИЯ СРОКОВ И СТОИМОСТИ

Исходные параметры
- Директивный срок T_dir : 38.83 дней
- Стоимость дня проекта S : 10 у.е.
- Исходная продолжительность : 52 дней
- Требуемое сокращение : 13.17 дней

Процесс оптимизации

Работа	Сокр.	Стоим.	Длит-ть

(1, 2)	5	20.00	47.00
(4, 6)	6	24.00	41.00
(2, 4)	2	10.85	38.83

ИТОГОВЫЕ ПОКАЗАТЕЛИ:

Продолжительность проекта : 38.83 дней
Стоимость сокращения : 54.85 у.е.
Общая стоимость проекта : 443.15 у.е.

Детализация сокращений

(1, 2) : -5.00 дн. → 20.00 у.е.
(4, 6) : -6.00 дн. → 24.00 у.е.
(2, 4) : -2.17 дн. → 10.85 у.е.